

Lead-Lag Detection in Crypto Markets

From Spectral Methods to Temporal Graph Neural Networks

Project Specification

December 31, 2025

Abstract

This project compares four approaches to discovering lead-lag relationships in cryptocurrency markets using tick-level trade data: (1) spectral analysis via the Hilbert transform, (2) Graph Attention Networks (GAT), (3) Temporal Graph Attention Networks (TGAT), and optionally (4) Sheaf Neural Networks. Unlike traditional equity markets, crypto trades 24/7 with publicly available tick data, making it ideal for studying high-frequency information propagation. We investigate whether BTC leads altcoins, how quickly price discovery propagates across assets, and whether learned attention patterns recover spectral phase relationships.

Contents

1	Motivation	2
1.1	Why Crypto?	2
1.2	The Core Question	2
1.3	What We'll Compare	2
2	Data	2
2.1	Asset Universe	2
2.2	Data Source	3
2.3	Data Volume	3
2.4	Preprocessing	3
2.5	Time Period	4
3	Method 1: Spectral Lead-Lag	4
3.1	Overview	4
3.2	Resolution Choice	4
3.3	Implementation	4
3.4	Expected Output	5
4	Method 2: GAT (Baseline)	5
4.1	Why Include It?	5
4.2	Setup	5
4.3	Architecture	6
4.4	Attention Extraction	6
5	Method 3: TGAT	7
5.1	Why TGAT Fits	7
5.2	Data Format	7
5.3	Two Formulations	7
5.4	Architecture	7

5.5	Temporal Attention Interpretation	8
6	Method 4: Sheaf Neural Networks (Optional)	8
6.1	The Idea	8
6.2	Why It's Hard	8
6.3	Implementation Sketch	8
6.4	When to Attempt	9
7	Comparison Analysis	9
7.1	Metric 1: Rank Correlation	9
7.2	Metric 2: Prediction Performance	9
7.3	Metric 3: Qualitative Analysis	10
7.4	Hypotheses	10
8	Implementation Plan	10
8.1	Week 1: Data Pipeline + Spectral	10
8.2	Week 2: GAT	10
8.3	Week 3: TGAT	11
8.4	Week 4: Comparison + Write-up	11
8.5	Week 5+ (Optional): Sheaf NN	11
9	Technical Stack	12
9.1	Installation	12
10	Expected Results	12
10.1	Spectral Ranking (Hypothesized)	12
10.2	Method Agreement (Hypothesized)	12
10.3	Prediction Performance (Hypothesized)	12
11	Success Criteria	13
12	References	13

1 Motivation

1.1 Why Crypto?

1. **Free tick data:** Binance provides historical trade-level data going back years, no payment required
2. **24/7 trading:** No overnight gaps, no market open/close effects
3. **Natural event structure:** Each trade is a discrete event, fitting TGAT's design perfectly
4. **Genuine lead-lag:** BTC is widely believed to lead altcoins; we can test this quantitatively
5. **Fast dynamics:** Lead-lag effects occur over seconds to minutes, not days

1.2 The Core Question

When BTC moves, how quickly do ETH, SOL, and other assets follow? Is this propagation structure:

- Recoverable from frequency-domain analysis (spectral)?
- Learnable by a contemporaneous attention model (GAT)?
- Better captured by temporal attention (TGAT)?
- Expressible as phase shifts in a sheaf structure?

1.3 What We'll Compare

Method	Temporal?	Learned?	Output	Fit to Tick Data
Spectral (Hilbert)	Yes (frequency domain)	No	Phases ϕ_{ij}	Good (after resampling)
GAT	No (contemporaneous)	Yes	Attention α_{ij}	Poor (snapshot-based)
GAT + lags	Implicit	Yes	Attention α_{ij}	Okay (hack)
TGAT	Yes (explicit)	Yes	Temporal attention	Native fit
Sheaf NN	Yes (by design)	Yes	Learned phases	Good (after resampling)

2 Data

2.1 Asset Universe

Ten liquid trading pairs on Binance (all against USDT):

Ticker	Name	Category	Expected Role
BTC	Bitcoin	Store of value	Leader
ETH	Ethereum	Smart contracts	Fast follower
BNB	Binance Coin	Exchange token	Exchange-specific
SOL	Solana	Alt L1	Follower
XRP	Ripple	Payments	Idiosyncratic
ADA	Cardano	Alt L1	Follower
DOGE	Dogecoin	Meme	Retail-driven
MATIC	Polygon	L2	ETH-correlated
LINK	Chainlink	Oracle	DeFi-correlated
AVAX	Avalanche	Alt L1	Follower

2.2 Data Source

Binance historical trade data: <https://data.binance.vision/>

```
1 # Download March 2024 trades for BTCUSDT
2 wget https://data.binance.vision/data/spot/daily/trades/BTCUSDT/BTCUSDT-trades
   -2024-03-01.zip
```

Each file contains:

Field	Description
trade_id	Unique trade identifier
price	Execution price
qty	Quantity traded
quote_qty	Quote asset quantity
time	Unix timestamp (milliseconds)
is_buyer_maker	Trade direction

2.3 Data Volume

Typical volumes:

- BTC/USDT: ~1–2 million trades/day
- ETH/USDT: ~500k–1M trades/day
- Smaller altcoins: ~100k–500k trades/day

For one week of data across 10 assets: ~50–100 million trades. We'll subsample.

2.4 Preprocessing

1. **Download:** Fetch daily ZIP files for each asset
2. **Parse:** Load into DataFrames, convert timestamps
3. **Aggregate:** Create uniform time bars (e.g., 1-second or 5-second bars)
4. **Align:** Ensure all assets have same timestamp index
5. **Compute returns:** Log returns for each bar

```
1 import pandas as pd
2 import numpy as np
3
4 def load_trades(filepath):
5     """Load Binance trade data from ZIP."""
6     df = pd.read_csv(
7         filepath,
8         compression='zip',
9         names=['trade_id', 'price', 'qty', 'quote_qty',
10              'time', 'is_buyer_maker', 'is_best_match']
11     )
12     df['time'] = pd.to_datetime(df['time'], unit='ms')
13     df = df.set_index('time')
14     return df
15
16 def aggregate_to_bars(trades, freq='5S'):
17     """Aggregate trades to OHLCV bars."""
18     bars = trades['price'].resample(freq).ohlc()
```

```

19 bars['volume'] = trades['qty'].resample(freq).sum()
20 bars['num_trades'] = trades['price'].resample(freq).count()
21 bars = bars.dropna()
22 return bars
23
24 def compute_returns(bars):
25     """Compute log returns from close prices."""
26     return np.log(bars['close'] / bars['close'].shift(1)).dropna()

```

2.5 Time Period

- **Development:** 1 week of data (March 1–7, 2024)
- **Full analysis:** 1 month if compute permits
- **Train/Val/Test:** 60%/20%/20% chronological split

3 Method 1: Spectral Lead-Lag

3.1 Overview

Same approach as before, but on higher-frequency data:

1. Aggregate trades to fixed bars (5-second)
2. Compute log returns
3. Apply Hilbert transform to get analytic signals
4. Compute complex correlation matrix
5. Extract global phases via Angular Synchronization

3.2 Resolution Choice

Bar Size	Bars/Day	Lead-Lag Horizon
1 second	86,400	Sub-second to seconds
5 seconds	17,280	Seconds to minutes
30 seconds	2,880	Minutes
1 minute	1,440	Minutes to hours

Recommendation: Start with **5-second bars**. Fine enough to capture fast dynamics, coarse enough to reduce noise.

3.3 Implementation

```

1 from scipy import signal
2
3 def compute_spectral_phases(returns_df):
4     """
5     Compute lead-lag phases via Hilbert transform.
6
7     returns_df: (T, N) DataFrame of log returns, columns are assets
8     Returns: (N,) array of global phases
9     """
10    # Analytic signals
11    analytic = pd.DataFrame(

```

```

12     signal.hilbert(returns_df.values, axis=0),
13     index=returns_df.index,
14     columns=returns_df.columns
15 )
16
17 # Standardize
18 Z = (analytic - analytic.mean()) / analytic.std()
19
20 # Complex correlation matrix
21 C = Z.values.conj().T @ Z.values / len(Z)
22
23 # Principal eigenvector
24 eigenvalues, eigenvectors = np.linalg.eigh(C)
25 v1 = eigenvectors[:, -1]
26
27 # Extract and center phases
28 theta = np.angle(v1)
29 theta -= np.angle(np.mean(np.exp(1j * theta)))
30
31 return theta, C

```

3.4 Expected Output

A ranking like:

Rank	Asset
1 (leads)	BTC
2	ETH
3	BNB
4	SOL
⋮	⋮
10 (lags)	DOGE

If BTC doesn't come out on top, that's interesting—maybe ETH leads in DeFi-correlated moves?

4 Method 2: GAT (Baseline)

4.1 Why Include It?

GAT isn't designed for temporal data, but it's a useful baseline:

- If GAT works well, maybe temporal structure isn't crucial
- If GAT fails, we have evidence that proper temporal modeling matters
- It's simple to implement and understand

4.2 Setup

Graph: Fully connected (10 nodes, 90 directed edges)

Node features at each time t (per bar):

- **ret_1:** Return this bar
- **ret_5:** Cumulative return over last 5 bars

- `ret_20`: Cumulative return over last 20 bars
- `vol_20`: Rolling volatility (20 bars)
- `volume_zscore`: Volume relative to recent average
- `num_trades`: Number of trades this bar

With lagged features (Option B from before):

- `ret_lag1`: Return of previous bar
- `ret_lag2`: Return two bars ago
- `neighbor_ret_lag1_mean`: Average neighbor return last bar

Target: Next-bar return sign (binary classification)

4.3 Architecture

```

1 import torch
2 import torch.nn.functional as F
3 from torch_geometric.nn import GATConv
4
5 class CryptoGAT(torch.nn.Module):
6     def __init__(self, in_channels, hidden_channels=32, heads=4):
7         super().__init__()
8         self.conv1 = GATConv(in_channels, hidden_channels, heads=heads, concat=
9             True)
10        self.conv2 = GATConv(hidden_channels * heads, hidden_channels, heads=1,
11            concat=False)
12        self.lin = torch.nn.Linear(hidden_channels, 1)
13
14    def forward(self, x, edge_index, return_attention=False):
15        x, (edge_index_1, alpha_1) = self.conv1(
16            x, edge_index, return_attention_weights=True
17        )
18        x = F.elu(x)
19        x = F.dropout(x, p=0.3, training=self.training)
20
21        x, (edge_index_2, alpha_2) = self.conv2(
22            x, edge_index, return_attention_weights=True
23        )
24        x = F.elu(x)
25
26        out = self.lin(x)
27
28        if return_attention:
29            return out, alpha_1
30        return out

```

4.4 Attention Extraction

After training, extract "who attends to whom":

```

1 def get_attention_ranking(model, data):
2     """Compute attention inflow for each node."""
3     model.eval()
4     with torch.no_grad():
5         _, alpha = model(data.x, data.edge_index, return_attention=True)
6
7     # alpha: (num_edges, num_heads) -> average over heads

```

```

8     alpha_mean = alpha.mean(dim=1)
9
10    # Sum attention flowing INTO each node
11    num_nodes = data.x.shape[0]
12    inflow = torch.zeros(num_nodes)
13    for i, (src, tgt) in enumerate(data.edge_index.T):
14        inflow[tgt] += alpha_mean[i]
15
16    return inflow.numpy()

```

5 Method 3: TGAT

5.1 Why TGAT Fits

TGAT was designed for exactly this: irregular events with timestamps. Each trade is an event, and TGAT can attend over historical events for each node.

5.2 Data Format

TGAT expects a stream of events, not snapshots:

Source	Target	Time	Features
BTC	BTC	1709312400.123	[price, volume, ...]
ETH	ETH	1709312400.456	[price, volume, ...]
BTC	BTC	1709312401.001	[price, volume, ...]

Self-edges represent "node i updated at time t ." Cross-edges can represent "price correlation links."

5.3 Two Formulations

Option A: Trade events only

- Each trade is an event on that asset's node
- Cross-asset edges connect correlated assets
- TGAT attends over each asset's recent trade history

Option B: Regular sampling with event structure

- Sample at fixed intervals (5 seconds)
- Treat each sample as an "event" for all nodes
- Simpler, but loses some temporal granularity

Recommendation: Start with Option B (simpler), then try Option A if time permits.

5.4 Architecture

```

1 # Using PyTorch Geometric Temporal
2 from torch_geometric_temporal.nn.attention import TGAT
3
4 class CryptoTGAT(torch.nn.Module):
5     def __init__(self, in_channels, hidden_channels=32, num_heads=4):
6         super().__init__()

```



```

7         self.tgat = TGAT(
8             in_channels=in_channels,
9             out_channels=hidden_channels,
10            heads=num_heads,
11        )
12        self.lin = torch.nn.Linear(hidden_channels * num_heads, 1)
13
14    def forward(self, x, edge_index, edge_time):
15        """
16        x: (num_nodes, in_channels) - node features
17        edge_index: (2, num_edges) - graph structure
18        edge_time: (num_edges,) - timestamp of each edge
19        """
20        h = self.tgat(x, edge_index, edge_time)
21        return self.lin(h)

```

5.5 Temporal Attention Interpretation

TGAT produces attention weights $\alpha_{ij}^{(t,\tau)}$: "how much does node i at time t attend to node j at time $\tau < t$?"

For lead-lag:

- High $\alpha_{\text{ETH} \leftarrow \text{BTC}}^{(t,t-1)}$: BTC's recent past matters for ETH now
- Compare $\alpha_{i \leftarrow j}$ vs $\alpha_{j \leftarrow i}$ to determine who leads

6 Method 4: Sheaf Neural Networks (Optional)

6.1 The Idea

Model lead-lag as phase shifts in a $U(1)$ sheaf:

- Each node has a complex-valued state
- Restriction maps are phase rotations $e^{i\phi_{ij}}$
- Learn the phases end-to-end

6.2 Why It's Hard

1. Complex tensor support in PyTorch is incomplete
2. Phases live on a circle, not $\mathbb{R}$ —gradient descent doesn't respect this
3. Antisymmetry constraint ($\phi_{ij} = -\phi_{ji}$) must be enforced
4. Initialization matters: random phases $\rightarrow$ no learning
5. Degenerate solutions (all phases $\rightarrow 0$) require careful regularization

6.3 Implementation Sketch

```

1 class SheafConv(torch.nn.Module):
2     def __init__(self, num_nodes, in_channels, out_channels):
3         super().__init__()
4         # Learnable phases (upper triangle only, for antisymmetry)
5         n_pairs = num_nodes * (num_nodes - 1) // 2
6         self.phase_params = torch.nn.Parameter(torch.zeros(n_pairs))
7         self.lin = torch.nn.Linear(in_channels, out_channels)

```

```

8
9     def get_phase_matrix(self, num_nodes):
10         """Build antisymmetric phase matrix."""
11         idx = torch.triu_indices(num_nodes, num_nodes, offset=1)
12         phases = torch.zeros(num_nodes, num_nodes)
13         phases[idx[0], idx[1]] = self.phase_params
14         phases[idx[1], idx[0]] = -self.phase_params
15         return phases
16
17     def forward(self, x, adj):
18         num_nodes = x.shape[0]
19         x = self.lin(x)
20
21         # Magnetic adjacency
22         phases = self.get_phase_matrix(num_nodes)
23         magnetic_adj = adj * torch.exp(1j * phases)
24
25         # Normalized diffusion
26         deg = adj.sum(dim=1)
27         deg_inv_sqrt = torch.diag(1.0 / torch.sqrt(deg + 1e-8))
28         norm_adj = deg_inv_sqrt @ magnetic_adj @ deg_inv_sqrt
29
30         # Aggregate (complex-valued)
31         x_complex = x.to(torch.cfloat)
32         out = norm_adj @ x_complex
33
34         return out.real

```

6.4 When to Attempt

Only after completing spectral, GAT, and TGAT. Budget 1–2 weeks for debugging.

7 Comparison Analysis

7.1 Metric 1: Rank Correlation

Compare lead-lag rankings across methods:

```

1 from scipy.stats import spearmanr
2
3 # Rankings from each method
4 spectral_rank = np.argsort(-spectral_phases) # Higher phase = leads
5 gat_rank = np.argsort(-gat_attention_inflow)
6 tgat_rank = np.argsort(-tgat_attention_inflow)
7
8 # Pairwise Spearman correlations
9 rho_spectral_gat, _ = spearmanr(spectral_rank, gat_rank)
10 rho_spectral_tgat, _ = spearmanr(spectral_rank, tgat_rank)
11 rho_gat_tgat, _ = spearmanr(gat_rank, tgat_rank)

```

7.2 Metric 2: Prediction Performance

Model	Task	Metrics
Baseline (logistic)	Predict next-bar sign	Accuracy, AUC
GAT	Predict next-bar sign	Accuracy, AUC
GAT + lags	Predict next-bar sign	Accuracy, AUC
TGAT	Predict next-bar sign	Accuracy, AUC

7.3 Metric 3: Qualitative Analysis

1. **Market Clock:** Polar plot of spectral phases
2. **Attention heatmaps:** GAT and TGAT attention matrices
3. **Lead-lag consistency:** Does BTC consistently lead across methods?
4. **Regime dependence:** Do rankings change during high volatility?

7.4 Hypotheses

Before running experiments:

1. BTC leads all other assets (spectral phase highest)
2. ETH is second (fast follower)
3. DOGE is most lagging (retail-driven, slow to react)
4. TGAT outperforms GAT on prediction (temporal structure matters)
5. Spectral and TGAT rankings correlate ($\rho > 0.6$)
6. GAT rankings correlate less with spectral ($\rho \approx 0.3\text{--}0.5$)

8 Implementation Plan

8.1 Week 1: Data Pipeline + Spectral

Day	Task
1	Write download script for Binance historical data
2	Load and parse trade data for 10 assets, one day
3	Aggregate to 5-second bars, align timestamps
4	Implement Hilbert transform pipeline, compute phases
5	Create Market Clock visualization, validate BTC leads

Deliverables:

- `data/trades/*.parquet` — raw trade data
- `data/bars_5s.parquet` — aggregated bars
- `results/spectral_phases.csv`
- `figures/market_clock.png`

8.2 Week 2: GAT

Day	Task
1	Create PyG Data objects from bar data
2	Implement GAT model, verify forward pass
3	Add lagged features (Option B)
4	Train GAT, extract attention weights
5	Compare attention ranking to spectral ranking

Deliverables:

- `models/gat.pt`
- `results/gat_attention.csv`
- `figures/attention_heatmap.png`

8.3 Week 3: TGAT

Day	Task
1	Install <code>torch_geometric_temporal</code> , understand API
2	Reformat data for TGAT (event stream or batched snapshots)
3	Implement TGAT model
4	Train TGAT, extract temporal attention
5	Compare TGAT ranking to spectral and GAT

Deliverables:

- `models/tgat.pt`
- `results/tgat_attention.csv`
- `figures/temporal_attention.png`

8.4 Week 4: Comparison + Write-up

Day	Task
1	Compute all comparison metrics (rank correlations, prediction accuracy)
2	Create comparison visualizations
3	Analyze disagreements between methods
4	Write final report
5	Clean code, documentation

Deliverables:

- `results/comparison_metrics.json`
- `figures/ranking_comparison.png`
- `report/final_report.pdf`

8.5 Week 5+ (Optional): Sheaf NN

Day	Task
1–2	Implement SheafConv with learnable phases
3	Handle complex gradients, phase projection
4	Initialize from spectral phases, train
5	Compare learned phases to spectral ground truth
6–7	Debug, regularize, iterate

9 Technical Stack

Component	Library
Data download	<code>requests</code> , <code>wget</code>
Data processing	<code>pandas</code> , <code>numpy</code> , <code>pyarrow</code>
Signal processing	<code>scipy.signal</code>
Graph neural networks	<code>torch_geometric</code>
Temporal GNNs	<code>torch_geometric_temporal</code>
Visualization	<code>matplotlib</code> , <code>seaborn</code> , <code>networkx</code>

9.1 Installation

```
1 pip install pandas numpy scipy matplotlib seaborn networkx pyarrow
2 pip install torch torch_geometric
3 pip install torch_geometric_temporal
```

10 Expected Results

10.1 Spectral Ranking (Hypothesized)

Rank	Asset	Rationale
1	BTC	Market leader, most liquidity
2	ETH	Second largest, DeFi hub
3	BNB	Exchange token, Binance-specific info
4	SOL	High-beta alt L1
5	LINK	DeFi infrastructure
6	AVAX	Alt L1
7	MATIC	ETH L2, follows ETH
8	ADA	Slower community
9	XRP	Idiosyncratic (legal issues)
10	DOGE	Retail meme coin, slowest

10.2 Method Agreement (Hypothesized)

	Spectral	GAT	TGAT
Spectral	1.0	0.4	0.7
GAT		1.0	0.5
TGAT			1.0

TGAT should agree more with spectral (both capture temporal structure) while GAT captures something different (contemporaneous importance).

10.3 Prediction Performance (Hypothesized)

Model	Accuracy	AUC
Baseline	0.50	0.50
GAT	0.51	0.52
GAT + lags	0.52	0.54
TGAT	0.53	0.56

Modest improvements—crypto is noisy and efficient. But TGAT should show consistent edge.

11 Success Criteria

The project is successful if:

1. **Data pipeline works:** Can download, parse, and aggregate Binance tick data
2. **Spectral method produces sensible ranking:** BTC leads or there’s a good explanation why not
3. **GAT and TGAT train without diverging:** Produce non-trivial attention patterns
4. **Comparison is quantified:** Rank correlations computed and interpreted
5. **You understand all four methods:** Can explain Hilbert transform, GAT attention, TGAT temporal attention, and sheaf phases

The project is *not* judged by:

- High prediction accuracy (crypto is efficient)
- Perfect method agreement (disagreement is interesting)
- Completing Sheaf NN (it’s optional)

12 References

References

- [1] Xu, D., et al. “Inductive Representation Learning on Temporal Graphs.” ICLR, 2020.
- [2] Veličković, P., et al. “Graph Attention Networks.” ICLR, 2018.
- [3] Bodnar, C., et al. “Neural Sheaf Diffusion.” NeurIPS, 2022.
- [4] Marple, S.L. “Computing the Discrete-Time Analytic Signal via FFT.” IEEE Trans. Signal Processing, 1999.
- [5] Aslanidis, N., et al. “Lead-Lag Relationships in Cryptocurrency Markets.” Finance Research Letters, 2022.